EXPERIENCES.FUN

Social Feed

Agent Implementation Specification

Purpose: Give the coding agent a precise product, UX, data-model, API, and implementation blueprint for a sophisticated activity-driven social feed. The feed should feel native to Experiences.fun and visually reuse the existing event-card language.

Core idea	The feed is a living stream of what people are doing, committing to, completing, and experiencing in the real world.
Primary objects	Experiences, status updates, accountability activity, achievements, contribution events, and community activity.
Design principle	Do not build a generic Twitter clone. Build an experience feed that happens to be social.
Implementation stance	Use existing experience cards, users, friends, XP, participation, attendance, completion, verification, and accountability infrastructure wherever possible.

Build target

A production-ready mobile feed that feels like a polished community activity surface rather than a stream of arbitrary posts. It should increase discovery of real-world experiences, make contribution visible, and give accountability activity a social context without exposing private information.

1. Product Definition

The Social Feed is the community's shared activity layer. It should answer one question immediately: **What is happening around me and among the people I care about?**

The feed is not primarily a content platform. User-generated posts are one activity type among several. The important object is **activity**: joining an experience, completing one, earning XP, achieving a milestone, checking in with an accountability partner, or sharing a structured status update.

Core loop

DISCOVER → JOIN → ATTEND → CONTRIBUTE → VERIFY → EARN XP → APPEAR IN FEED → SOCIAL VALIDATION → DISCOVER NEXT EXPERIENCE

Product rules

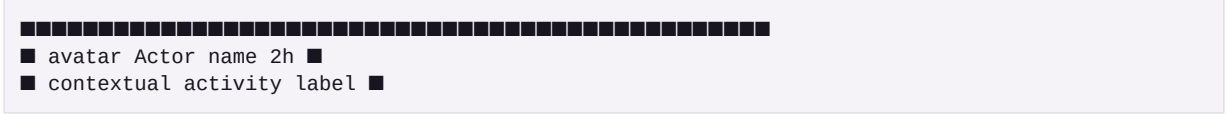
- Experience cards remain the hero object and should reuse the existing event-card visual language.
- Feed items should be visually rich, compact, scannable, and consistent with the rest of the app.
- Do not make every backend event visible. Only meaningful, socially useful activity should enter the feed.
- Do not reward likes/comments with XP. XP should represent real contribution or meaningful product actions.
- Keep private accountability information private unless an explicit visibility setting allows sharing.
- Do not turn the feed into a purely chronological firehose. Use deterministic relevance ranking for the initial version.
- Notifications remain separate. Feed = what is happening. Notifications = what requires my attention.

2. Feed Content Types

Type	Example	Rendering
Experience	Sarah joined Hackney Park Cleanup	Nested existing experience card
Status	Working toward my first community project this month	Structured status card
Completed	Alex completed Intro to Web3	Completion / achievement card
Accountability	Sam completed 4/5 weekly commitments	Progress card + encouragement CTA
Achievement	Maya unlocked Community Builder	Achievement card
Contribution	Jordan earned 50 XP for verified participation	Contribution / XP card
Photo / Post	Photo from today's community event	Media/status card

3. Feed Card Anatomy

Every card should feel like part of one visual system, even when the underlying event differs.



```
■ ■
■ [ Nested content / experience card ] ■
■ ■
■ location / category / XP / progress ■
■ ■
■ ■ 12 ■ 4 ■ Share ■
■■■
```

```
[ Encourage Alex ] [ View progress ]

--------------------------------

■ ACCOUNTABILITY
Sam checked in with you

"How's the coding challenge going?"

[ Reply ]
```

5. Technical Architecture

Implement the feed as a reusable feature module rather than adding more logic to the already-large main navigation/screen file. The feed should consume existing product primitives rather than duplicate experience, user, XP, friendship, notification, or accountability logic.

Suggested mobile structure

```
components/feed/
Feed.tsx
FeedItem.tsx
ExperienceFeedCard.tsx
StatusFeedCard.tsx
AccountabilityFeedCard.tsx
AchievementFeedCard.tsx
ContributionFeedCard.tsx
ReactionBar.tsx
CommentPreview.tsx
CreateStatus.tsx
FeedFilters.tsx

hooks/
useFeed.ts
useCreateStatus.ts
usePostReaction.ts
useComments.ts
```

Suggested server/domain model

```
activity_events
---------------
id
actor_id
type
target_id
target_type
metadata
visibility
created_at

post_reactions
--------------
id
post_id
user_id
reaction_type
created_at

comments
--------
id
post_id
user_id
body
created_at

posts
-----
id
author_id
body
media_url
visibility
created_at
```

Activity event types

- experience_joined
- experience_completed
- experience_verified
- xp_earned
- achievement_unlocked
- accountability_checkin
- accountability_goal_completed
- friend_added
- status_posted

Visibility

- public - eligible for broad community discovery
- friends - visible to accepted friends / relevant social graph
- community - visible within a relevant community
- private - never surfaced into the public feed

Important: accountability events should default to privacy-safe behavior. A private goal, private check-in, or direct message must not accidentally become a public feed event.

6. API / Data Contract

Initial endpoints

```
GET /api/feed
POST /api/posts
GET /api/posts/:id
POST /api/posts/:id/reactions
DELETE /api/posts/:id/reactions
GET /api/posts/:id/comments
POST /api/posts/:id/comments
DELETE /api/comments/:id
```

GET /api/feed parameters

```
mode=for_you|following|local
cursor=<pagination cursor>
limit=20
communityId=<optional>
```

Feed response shape

```
{
"items": [
{
"id": "activity-id",
"type": "experience_joined",
"actor": {...},
"target": {...},
"metadata": {...},
"visibility": "friends",
"createdAt": "..."
}
],
"nextCursor": "..."
}
```

Implementation notes

- Prefer cursor pagination over offset pagination.
- Return enough hydrated data to render the first card without a cascade of network requests.
- Reuse existing API types and generated query keys where available.
- Use React Query / TanStack Query for caching, pagination, invalidation, and optimistic reactions.
- Use deterministic ranking before introducing ML.

7. Feed Ranking

Initial ranking can be simple and explainable.

Signal	Relative importance
Direct friend / followed user	High
Nearby/local relevance	High
Experience relevance to user	High
Freshness	Medium-high
Community relevance	Medium
Engagement	Low-medium

Do not optimize for raw engagement at the expense of contribution. A post with 100 likes should not automatically outrank a nearby experience that is genuinely relevant to the user.

8. Integration With Existing Experiences.fun Systems

- **Experiences:** joining, attendance, completion, verification, host activity, and new experiences can generate feed events.
- **XP:** meaningful XP awards can surface as contribution events or achievements.
- **Friends:** friendship relationships should influence Following and For You ranking.
- **Accountability:** progress and check-ins can become feed cards only when visibility rules permit.
- **Notifications:** remain separate and action-oriented.
- **Profiles:** tapping an actor should open the existing profile/friend profile surface.
- **Explore:** feed cards should link naturally into experience discovery.

9. Example Feed Session

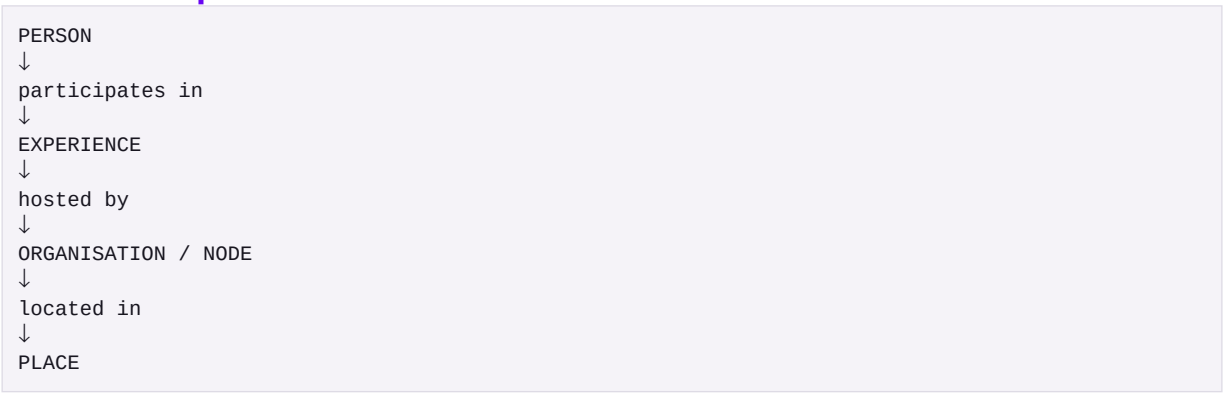
```
FOR YOU

Sarah joined an experience
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■ COMMUNITY ■
■ Hackney Park Cleanup ■
■ Sat 10:00 AM · Hackney ■
■ 18 / 25 participants ■
```

```
■ [ View experience ] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 8 ■ 2

Alex completed an experience
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ✓ COMPLETED ■
■ Intro to Web3 ■
■ +50 XP ■
■ [ View experience ] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ 14 ■ 3

■ ACCOUNTABILITY
Maya completed 4/5 weekly commitments
"One more to go."
[ Encourage Maya ]

NALI posted a status
"Trying to complete my first 3 community
projects this month."
■ 5 ■ 1

NEW EXPERIENCE
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ ■ EDUCATION ■
■ Community Coding Workshop ■
■ Sun 2:00 PM · Tottenham ■
■ [ Join experience ] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
```

10. Activity Graph & Feed Architecture

The long-term architecture should treat the feed as a **visualization of Experiences.fun's activity graph**, rather than a standalone social product. This is the key architectural idea behind the feed.

Core relationship model



This means the feed can eventually answer richer questions than a conventional social feed: who is active, what experiences are happening, which organisations are contributing, what communities are moving, and where measurable impact is being generated.

11. Feed Density & Activity Aggregation

The feed should feel curated rather than like a database query wearing a UI.

- Avoid repeated activity from the same actor within a short window unless the activities are meaningfully different.
- Collapse repeated participation in the same experience into aggregated activity where appropriate.
- Example: **Alex, Sarah, Maya + 12 others joined Hackney Park Cleanup.**
- Aggregate community-level achievements such as **5 people completed experiences this week**.
- Use activity deduplication and grouping before ranking the final feed.

12. Activity Relevance Reasons

For You ranking should be explainable and debuggable. Store internal relevance reasons for each feed item.

```
{
"relevanceReasons": [
"friend_activity",
"nearby",
"similar_experience",
"followed_host"
]
}
```

These reasons do not need to be exposed in the first UI release, but they provide a clean foundation for debugging ranking and eventually showing lightweight explanations such as **Because you follow HydeOut Studios** or **Popular near you**.

13. Social Proof Without Vanity Metrics

Prioritize evidence of real-world participation over attention metrics.

- People participating
- People completing
- XP earned through meaningful contribution
- Experiences attended
- Community milestones
- Friends participating

Avoid making follower counts, raw impressions, or engagement farming the central social currency. The feed should communicate that people are doing things, not merely staring at their phones.

14. Personal Progress Insertions

Occasionally insert a personalized progress card into the feed to connect social activity with personal reflection.

```
YOUR WEEK

■ 3 experiences
■ 2 commitments completed
■ +180 XP
■ 4 people influenced

You're building momentum.
```

This should be an occasional, useful insertion rather than an advertisement for the user's own statistics every five seconds.

15. Community Milestones

Community-wide milestones can make collective impact visible.

```
■ COMMUNITY MILESTONE

Experiences.fun members have completed
1,000 experiences.

1,000 / 1,000

Keep going.
```

Other examples include education hours completed, volunteer participation, environmental actions completed, or local community goals reached. These events can become powerful bridges between activity, impact, and reputation.

16. Agent Acceptance Criteria

The implementation is complete when all of the following are true:

- The Home/Feed surface renders a mixed stream of meaningful activity rather than only user posts.
- Experience activity uses the existing event-card visual language.
- A user can create a status update from the feed.
- Status updates appear as feed activity and persist correctly.
- Experience join/completion/verification events can appear in the feed.
- Meaningful XP/achievement events can appear in the feed.
- Accountability progress/check-in cards can appear when privacy rules permit.
- Private accountability information is never accidentally exposed.
- For You, Following, and Local modes work.
- Feed pagination works without duplicate or missing items.
- Reactions and comments update optimistically and invalidate correctly.
- Tapping an experience opens the existing experience detail surface.
- Tapping a user opens the existing profile/friend profile surface.
- Feed and notification behavior remain conceptually separate.
- The feed is implemented as modular components/hooks rather than adding a giant new block to the main navigation file.
- Existing authentication, permissions, XP, friendship, experience, and accountability systems remain the source of truth.
- Loading, empty, error, and offline-ish states are handled gracefully.
- The UI is responsive and visually consistent with the existing Experiences.fun mobile app.
- Repeated events are aggregated where appropriate.
- Community milestones and personal progress insertions have clear visibility rules.
- The feed architecture is extensible through activity event types rather than hard-coded feed-only logic.

17. MVP Build Order

1	Create feed/activity data model	Add posts + activity_events + reactions/comments as needed; add visibility.
2	Create feed API	Implement cursor pagination, hydration, visibility filtering, aggregation, and deterministic ranking.
3	Build FeedItem abstraction	Map event type → specific card renderer.
4	Build experience feed card	Reuse the existing experience-card design and interaction patterns.
5	Build status card + composer	Keep the first composer intentionally small.

6	Build accountability card	Only expose privacy-safe summary/progress events.
7	Add reactions/comments	Optimistic UI with query invalidation.
8	Add feed modes	For You / Following / Local.
9	Connect automatic events	Join, complete, verify, XP, achievement, accountability.
10	Add aggregation + milestones	Collapse repetitive activity and introduce community/personal progress cards.
11	Polish + test	Loading, empty, pagination, permissions, accessibility, performance.

18. Non-Goals

- Do not build a generic Twitter/X clone.
- Do not build a full media editing suite in the MVP.
- Do not introduce an opaque ML recommendation system before the deterministic system is working.
- Do not expose every database event as a feed item.
- Do not mix private notifications and public social activity.
- Do not create a separate parallel experience system just for the feed.
- Do not award XP for vanity engagement such as likes.
- Do not let engagement metrics become the primary definition of community value.

19. Final Product Principle

Don't make the feed about content. Make it about activity. Content is merely one type of activity.

The finished feed should make Experiences.fun feel like a living map of contribution: people discovering experiences, showing up, helping, learning, progressing, holding each other accountable, and making that activity visible. The social layer should reinforce the real-world layer, not compete with it.

The feed is ultimately the human-readable interface for the underlying social and impact graph. That is what makes it strategically different from another generic social feed.

Implementation should favor reuse, composability, privacy-safe defaults, deterministic behavior, and a strong visual relationship between feed cards and the existing experience cards. Humans have already invented enough social feeds. This one should have a reason to exist.